DOCUSTREAM WEB – A SERVERLESS DOCUMENT PROCESSING AND METADATA INTELLIGENCE PLATFORM

A MINI PROJECT REPORT

Submitted by

BENIYAL J – 311124205010

NAVINKUMAR S – 311124205304

DANIEL RAJ V – 311124205012

In partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY

LOYOLA-ICAM COLLEGE OF ENGINEERING AND TECHNOLOGY

ANNA UNIVERSITY: CHENNAI 600 025

NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that the Miniproject report on "**DOCUSTREAM WEB – A SERVERLESS DOCUMENT PROCESSING AND METADATA INTELLIGENCE PLAFORM"** is the bonafide work of **BENIYAL J (311124205010), DANIEL RAJ V (311124205012), NAVINKUMAR S (311124205304)** who carried out the project work under my supervision.

SIGNATURE

Ms. Abinaya,
B.E., M.E.
Project Coordinator
Department of Information Technology
Loyola-ICAM College of Engineering and Technology
Nungambakkam, Chennai-34

SIGNATURE

Ms. Sherril Sophie Maria Vincent
B.E., M.E. , Ph.D.
Head of the Department
Department of Information Technology
Loyola-ICAM College of Engineering and Technology
Nungambakkam, Chennai-34

ACKNOWLEDGEMENT

First of all, we thank The Lord Almighty for showering His blessings on us, which enabled us to complete this project successfully.

We express our sincere gratitude to our beloved and respected Director, **Rev. Dr. Justine SJ**. We wish to acknowledge with thanks to our college Dean of Students, Rev. **Dr. Albert William SJ** for his support and encouragement.

We extend our sincere gratitude to **Dr. A. Baskar**, Principal, for his motivation and support towards this project.

We express our sincere gratitude to the Head of the Department of Information Technology, **Ms. Sherril Sophie Maria Vincent** for her constant support, motivation, encouragement and being a source of inspiration throughout this project and during the course of this semester.

Our sincere thanks to our Project Coordinator **Ms. Abinaya** for her valuable guidance in shaping this project.

We also take this opportunity to thank all the Faculty and Non-teaching staff members of the Department of Information Technology for their constant support. Finally, we thank each and every one who helped us to complete this project.

BENIYAL J

DANIEL RAJ V

NAVIN KUMAR S

ABSTRACT

DocuStream Web – A Serverless Document Processing and Metadata Intelligence Platform is a cloud-native web application that lets users upload business documents (PDF, PNG and JPEG), automatically extracts their text using optical character recognition (OCR), and derives structured metadata such as email addresses, phone numbers and keywords. The system is built entirely on serverless Amazon Web Services (AWS) components: Amazon S3 for object storage, AWS Lambda (Python 3.11) for compute, AWS Textract for OCR, Amazon DynamoDB for metadata, and Amazon API Gateway for the REST interface. The whole infrastructure is declared as code using the AWS Serverless Application Model (SAM). The user interface is a Next.js 14 application written in TypeScript with Tailwind CSS and deployed on Vercel.

A key design decision is direct-to-S3 ingestion using presigned URLs: the browser uploads the file straight to S3, so binary data never passes through API Gateway or Lambda. When the upload completes, an S3 event triggers a Lambda function that runs Textract, parses entities with regular expressions and stop-word-filtered keyword ranking, and stores the result in DynamoDB. A single-table DynamoDB design keeps each user's storage metrics and document records in one partition.

The platform enforces a 20 MB single-file limit and a 100 MB per-user storage quota, supports inline preview and forced download through separate presigned GET URLs, and reclaims quota when a document is deleted. A backend test suite of eleven pytest/moto tests passes, and automated scripts deploy the backend to AWS and the frontend to Vercel. DocuStream Web demonstrates the pay-per-use, event-driven and zero-server-administration characteristics of serverless cloud architecture. It is an educational prototype and is not intended to replace production document-management systems.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iv
	LIST OF FIGURES	vii
	LIST OF TABLES	viii
	LIST OF ABBREVIATIONS	ix
1	INTRODUCTION	1
1.1	Introduction	1
1.2	Problem Statement	1
1.3	Objectives	2
1.4	Scope	2
2	REQUIREMENTS	3
2.1	Functional Requirements	3
2.2	Non-Functional Requirements	3
2.3	Hardware Requirements	4
2.4	Software Requirements	4
3	ANALYSIS	5
3.1	Existing System	5
3.2	Proposed System	5
3.3	Bull Diagram	6
3.4	Octopus Diagram	7
4	SYSTEM DESIGN	9
4.1	System Architecture	9
4.2	Data Flow Diagram	9
4.3	Use Case Diagram	10
4.4	Component Diagram	11
4.5	Sequence Diagram	12
4.6	Activity Diagram	13
4.7	Data Model (Class/Entity Diagram)	14
5	MODULE DESCRIPTION	15
5.1	Presigned URL Module	15

CHAPTER NO.	TITLE	PAGE NO.
5.2	Document Processing Module	15
5.3	Document Listing Module	16
5.4	Document Deletion Module	16
5.5	Infrastructure Module	16
5.6	Web User Interface Module	16
6	IMPLEMENTATION	17
6.1	Infrastructure as Code	17
6.2	Backend Implementation	17
6.3	Frontend Implementation	18
6.4	Data Model and Access Patterns	19
6.5	Quota Management	19
6.6	Security Implementation	19
6.7	Deployment	20
7	TESTING AND VERIFICATION	21
7.1	Unit Testing	21
7.2	Functional Testing	21
7.3	Integration Testing	22
7.4	Cloud and Security Testing	22
8	CONCLUSION AND FUTURE ENHANCEMENTS	24
8.1	Conclusion	24
8.2	Limitations	24
8.3	Future Enhancements	25
9	REFERENCES	26
10	APPENDIX	27
A.	Project Structure	27
B.	Important Commands	27
C.	Representative Data Records	28
D.	Screenshots	29

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	Bull Diagram	7
3.2	Octopus Diagram	8
4.1	System Architecture	9
4.2	Data Flow Diagram	10
4.3	Use Case Diagram	11
4.4	Component Diagram	12
4.5	Sequence Diagram	13
4.6	Activity Diagram	13
4.7	Data Model (Entity Diagram)	14
6.1	Document Lifecycle	20
D.1	DocuStream Web dashboard and upload area	29
D.2	Processed image with extracted metadata	29
D.3	Document viewer showing pending upload status	30
D.4	OCR text and metadata extraction result	30

LIST OF TABLES

Table No.	Title	Page No.
2.1	Tools and Technologies Used	4
3.1	Comparison of Existing and Proposed Systems	5
5.1	Major Project Modules	15
6.1	REST API Endpoints	19
6.2	Lambda Function Configuration	17
6.3	DynamoDB Access Patterns	19
7.1	Automated Test Cases	21
7.2	Manual Verification Checklist	22

LIST OF ABBREVIATIONS

S. No.	Abbreviation	Expansion
1	API	Application Programming Interface
2	AWS	Amazon Web Services
3	ARN	Amazon Resource Name
4	CLI	Command Line Interface
5	CORS	Cross-Origin Resource Sharing
6	CSS	Cascading Style Sheets
7	DDB	Amazon DynamoDB
8	HTTP / HTTPS	Hypertext Transfer Protocol / Secure
9	IAM	Identity and Access Management
10	IaC	Infrastructure as Code
11	JSON	JavaScript Object Notation
12	JWT	JSON Web Token
13	OCR	Optical Character Recognition
14	PK / SK	Partition Key / Sort Key
15	REST	Representational State Transfer
16	S3	Amazon Simple Storage Service
17	SAM	Serverless Application Model
18	SigV4	AWS Signature Version 4
19	SSE	Server-Side Encryption
20	UUID	Universally Unique Identifier

CHAPTER 1: INTRODUCTION

1.1 INTRODUCTION

Organisations in education, law, finance and healthcare receive large numbers of documents every day, usually as PDF files or scanned images. Extracting useful information from them, such as contact details, reference numbers and key terms, is normally done by people reading each file and typing the data into another system. Hosting such a processing service on always-on servers adds cost and operational work.

DocuStream Web – A Serverless Document Processing and Metadata Intelligence Platform is developed as a Cloud Application Development mini project that shows a different approach. The user uploads a document through a web dashboard. The file goes directly to Amazon S3 using a short-lived presigned URL. S3 then triggers an AWS Lambda function that reads the file, calls AWS Textract to perform OCR, and applies regular expressions and keyword ranking to extract emails, phone numbers and keywords. The results are stored in Amazon DynamoDB and shown in the web interface.

The backend consists of four Python 3.11 Lambda functions exposed through Amazon API Gateway, all defined in a single AWS SAM template. The frontend is a Next.js 14 application with TypeScript and Tailwind CSS, hosted on Vercel.

The project connects several cloud concepts into one working application: event-driven computing, object storage, managed OCR, NoSQL single-table design, infrastructure as code, least-privilege IAM, and pay-per-use billing. DocuStream Web therefore combines cloud services, web development and automation scripts to create a lightweight, scalable document intelligence system.

1.2 PROBLEM STATEMENT

Traditional document intake systems have three main weaknesses:

- **Manual data entry and delay.** Staff must open each document, find the relevant details and transcribe them, which is slow and error-prone.
- **Idle cost and limited scalability.** A server-based service running on virtual machines or container clusters must be provisioned all day, costs money when idle, and needs extra orchestration to handle sudden upload spikes.
- **Bandwidth pressure on application servers.** When large files are uploaded through the application server, they consume connections, CPU and memory, which can lead to timeouts and failures under concurrent use.

When such a service is opened to the public, there is also a risk of misuse through very large files or unlimited storage. There is therefore a need for an application that can accept documents without burdening a server, extract their text and key entities automatically, and enforce limits on file size and per-user storage.

The project aims to develop a document platform that can:

- Accept PDF, PNG and JPEG uploads directly into object storage without passing binary data through backend servers.
- Run OCR automatically when an upload completes.
- Extract emails, phone numbers and keywords from the recognised text.
- Store the results in a database and display them in a searchable web interface.
- Limit each file to 20 MB and each user to 100 MB of total storage.
- Provide both inline preview and direct download of stored documents.

- Delete a document completely and give the storage quota back to the user.
- Be deployed and updated through automated scripts, with no servers to manage.

1.3 OBJECTIVES

The main objectives of the DocuStream Web project are:

1. To develop a serverless document processing platform on AWS using Lambda, S3, Textract, DynamoDB and API Gateway.
2. To implement direct-to-S3 uploads using presigned PUT URLs so that file bytes bypass the application tier.
3. To trigger OCR and entity extraction automatically through S3 event notifications.
4. To design a single-table DynamoDB schema that stores user storage metrics and document records in one partition.
5. To enforce a 20 MB single-file limit and a 100 MB per-user storage quota.
6. To provide secure inline preview and download of documents through presigned GET URLs.
7. To build a responsive Next.js dashboard with drag-and-drop upload, storage meter, search and an OCR viewer.
8. To define the entire infrastructure as code using AWS SAM with least-privilege IAM policies.
9. To verify the backend with an automated pytest and moto test suite.
10. To automate deployment of the backend to AWS and the frontend to Vercel.

1.4 SCOPE

The scope of DocuStream Web includes uploading documents, running OCR, extracting emails, phone numbers and keywords, storing results, listing and searching documents, previewing and downloading files, deleting documents, and tracking per-user storage.

The system supports three file types (`application/pdf`, `image/png`, `image/jpeg`), a 20 MB single-file ceiling (20,971,520 bytes), a 100 MB per-user quota (104,857,600 bytes), and a local mock mode in which the frontend runs without any AWS resources.

The project is intended as an educational prototype and a public demonstration. Several boundaries apply: users are selected from a demo switcher rather than authenticated, synchronous Textract is suited to single-page documents and images, entity extraction is rule-based rather than machine-learned, and the system is not designed for production document-management workloads.

CHAPTER 2: REQUIREMENTS

2.1 FUNCTIONAL REQUIREMENTS

1. **Upload Validation.** The system shall accept only PDF, PNG and JPEG files of at most 20 MB, checking this in the browser and again in the backend.
2. **Presigned URL Issuance.** The system shall issue a 15-minute presigned S3 PUT URL after validating file size and the user's remaining quota.
3. **Direct Upload.** The browser shall upload the file directly to S3 and show upload progress.
4. **Automatic OCR.** The system shall run AWS Textract automatically when an object is created under the `uploads/` prefix.
5. **Entity Extraction.** The system shall extract email addresses, phone numbers and the most frequent keywords from the recognised text.
6. **Metadata Persistence.** The system shall store the extracted text, parsed metadata, processing status and timestamps in DynamoDB.
7. **Quota Tracking.** The system shall maintain each user's total storage bytes and document count, and shall reject uploads that would exceed 100 MB.
8. **Document Listing.** The system shall list all documents for a user with their status, along with preview and download URLs and quota metrics.
9. **Preview and Download.** The system shall offer an inline preview and a forced-download link for each document.
10. **Search and Filter.** The interface shall search file names, OCR text, emails, phones and keywords, and filter by status.
11. **Deletion.** The system shall delete the S3 object and database record, and reduce the user's storage count accordingly.
12. **Fallback Extraction.** If Textract is unavailable in the account, the system shall fall back to a local text extractor instead of failing.

2.2 NON-FUNCTIONAL REQUIREMENTS

Performance Uploads should not consume backend bandwidth, and OCR results should appear in the dashboard shortly after upload, with the interface polling every 2 seconds.

Scalability Compute, storage and database layers should scale automatically with load. DynamoDB uses on-demand capacity and Lambda scales per invocation.

Cost Efficiency The system should have no baseline idle cost; billing should depend only on actual use.

Security The S3 bucket shall be private with all public access blocked, data shall be encrypted at rest, and all file access shall use time-limited SigV4 presigned URLs. Each Lambda function shall receive only the permissions it needs.

Reliability The system should degrade gracefully when Textract is not available, and should keep point-in-time recovery enabled on the database.

Usability The interface should be responsive and show clear progress, quota and status information with drag-and-drop upload.

Maintainability Backend functions, shared utilities, infrastructure template, frontend components and deployment scripts should be separate modules. Infrastructure should be fully reproducible from the SAM template.

Portability The frontend should work against either the real API Gateway or a local mock API by changing one environment variable.

2.3 HARDWARE REQUIREMENTS

- Development machine: computer or laptop (minimum 4 GB RAM)
- Active internet connection for AWS and Vercel deployment
- Users: any computer, tablet or smartphone with a modern web browser

2.4 SOFTWARE REQUIREMENTS

- Operating System: Windows / Linux / macOS
- Python 3.11 (Lambda runtime) with boto3, pytest and moto
- Node.js and npm for the Next.js 14 frontend
- AWS account, AWS CLI and AWS SAM CLI
- Vercel account and Vercel CLI
- Modern web browser (Chrome, Edge, Firefox, Safari)
- Git and GitHub for version control
- Visual Studio Code or equivalent development environment

Table 2.1 – Tools and Technologies Used

Technology	Role in the Project
AWS SAM (CloudFormation)	Infrastructure as code and deployment of the backend stack
Amazon S3	Private object storage for uploaded documents; emits upload events
AWS Textract	Managed OCR (detect_document_text)
AWS Lambda (Python 3.11)	Four serverless microservices
Amazon DynamoDB	Single-table store for metrics and document records
Amazon API Gateway	Regional REST API with CORS
AWS IAM	Least-privilege roles for each function
Next.js 14 (App Router)	Frontend framework and local mock API routes
TypeScript	Typed frontend code and API contracts
Tailwind CSS	Responsive user interface styling
Axios	HTTP client with upload progress tracking
Vercel	Hosting for the Next.js frontend
pytest and moto	Backend unit testing with in-memory AWS mocks
Git / GitHub	Version control and collaboration

CHAPTER 3: ANALYSIS

3.1 EXISTING SYSTEM

In a conventional document intake system, a monolithic web application runs on virtual machines or containers. Users upload files to the application server, which saves them, runs processing code and writes to a database. Often the processing is limited to storing the file, and a person still reads each document to find the data. This approach has the following limitations:

- Heavy binary uploads pass through the application server, consuming bandwidth, memory and connections.
- Servers must run around the clock, so there is a fixed monthly cost even when nobody uses the system.
- Handling sudden bursts of uploads needs auto-scaling groups with warm-up delays and manual capacity planning.
- Operating system patching, runtime updates and monitoring add administrative work.
- Contact details and keywords are usually entered by hand, which is slow and error-prone.
- Without quotas, a public service can be abused through very large or unlimited uploads.

3.2 PROPOSED SYSTEM

DocuStream Web uses an event-driven serverless model. The browser requests a presigned URL from a Lambda function, uploads the file straight to S3, and S3 triggers a second Lambda function that performs OCR and entity extraction. The working of the system is:

- The user drops a file on the dashboard and the browser checks its type and size.
- The browser requests an upload URL; the backend checks file size and remaining quota, creates a `PENDING_UPLOAD` record and returns a 15-minute presigned PUT URL.
- The browser uploads the file directly to S3 with a progress bar.
- S3 raises an `ObjectCreated` event that invokes the processor Lambda function.
- The processor reads the file, calls Textract (or the local fallback), extracts emails, phones and keywords, marks the record `PROCESSED` and increments the user's storage usage.
- The dashboard polls the list API, and the document appears with its metadata and preview and download links.
- The user may delete a document, which removes the file and record and reclaims the quota.

This separates the data path (browser to S3) from the control path (API Gateway and Lambda). The intended users are students demonstrating serverless concepts, small teams that need quick OCR and metadata extraction, and developers learning event-driven cloud design.

Table 3.1 – Comparison of Existing and Proposed Systems

Feature	Traditional Server-Based System	DocuStream Web
Idle infrastructure cost	Always-on VMs or containers	No cost when idle (pay per use)
Upload path	Through the application server	Direct to S3 using presigned URL
Scaling	Auto-scaling groups with warm-up	Automatic Lambda and on-demand DynamoDB scaling
Text extraction	Manual or custom OCR service	Managed Textract with local fallback
Metadata extraction	Manual entry	Automatic emails, phones, keywords

Feature	Traditional Server-Based System	DocuStream Web
Database scaling	Provisioned capacity	DynamoDB PAY_PER_REQUEST
Server maintenance	OS patches and runtime updates	None; managed by AWS
Storage limits	Often unenforced	20 MB per file and 100 MB per user
Infrastructure setup	Manual or scripted	Reproducible SAM template

3.3 BULL DIAGRAM

The Bull Diagram represents the central problem of processing uploaded documents at scale, together with its causes and effects.

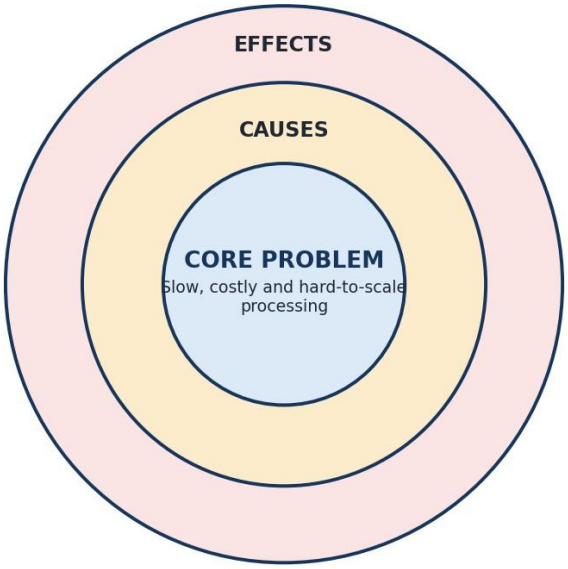
Central Problem Slow, costly and hard-to-scale intake and processing of unstructured documents

Major Causes

- Manual reading and data entry of document contents
- Binary uploads routed through application servers
- Always-on infrastructure that must be provisioned for peak load
- Scanned images and PDFs have no searchable text
- No built-in limits on file size or per-user storage
- Manual server patching and capacity management

Effects

- Delays and transcription errors in data entry
- Server timeouts and crashes under concurrent uploads
- High baseline cost even when idle
- Documents that cannot be searched by content
- Risk of storage abuse on public deployments
- Ongoing maintenance effort for administrators



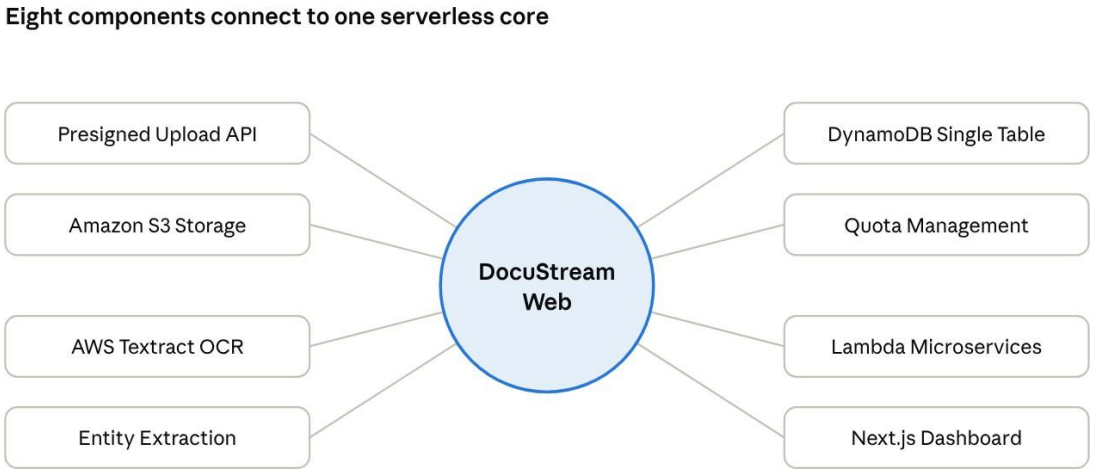
Bullseye view: the core problem is surrounded by its causes and wider effects.

Figure 3.1 – Bull Diagram

3.4 OCTOPUS DIAGRAM

The central system is DocuStream Web – A Serverless Document Processing and Metadata Intelligence Platform. The major components connected to it are:

- Presigned Upload Service
- Amazon S3 Document Storage
- AWS Textract OCR
- Entity Extraction Engine
- DynamoDB Single-Table Store
- Quota Management
- API Gateway and Lambda Microservices
- Next.js Web Dashboard



octopus diagram: core system with 8 components

Figure 3.2 – Octopus Diagra

CHAPTER 4: SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

The DocuStream Web architecture consists of six layers: the Next.js client on Vercel, the API Gateway tier, the Lambda compute tier, the S3 storage tier, the Textract document-AI tier, and the DynamoDB database tier. The design deliberately separates the **control path** (small JSON requests through API Gateway and Lambda) from the **data path** (the file bytes, which travel only between the browser and S3).

The upload flow is:

Browser → HTTPS → API Gateway → PresignedUrlFunction → DynamoDB (quota check, pending record) → presigned PUT URL → Browser → HTTPS PUT → Amazon S3

The processing flow is:

Amazon S3 (`ObjectCreated` on `uploads/`) → DocumentProcessorFunction → AWS Textract → entity extraction → DynamoDB (record `PROCESSED`, quota incremented)

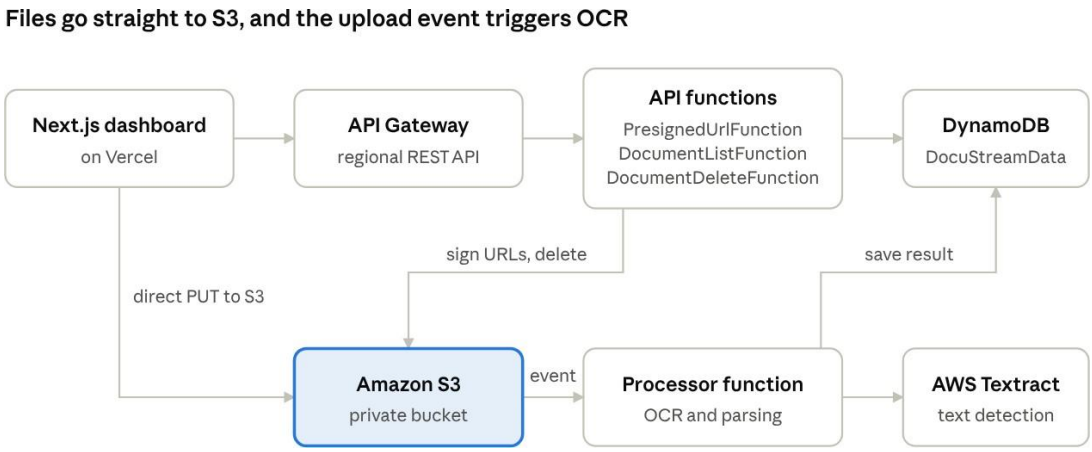
The retrieval flow is:

Browser → HTTPS → API Gateway → DocumentListFunction → DynamoDB → presigned preview and download URLs → Browser

The deletion flow is:

Browser → HTTPS → API Gateway → DocumentDeleteFunction → S3 (delete object) and DynamoDB (delete record, reduce quota)

The S3 bucket is private, and every file access uses a SigV4 presigned URL, so the browser never needs AWS credentials.



system architecture: 6 components, direct-to-S3 upload path

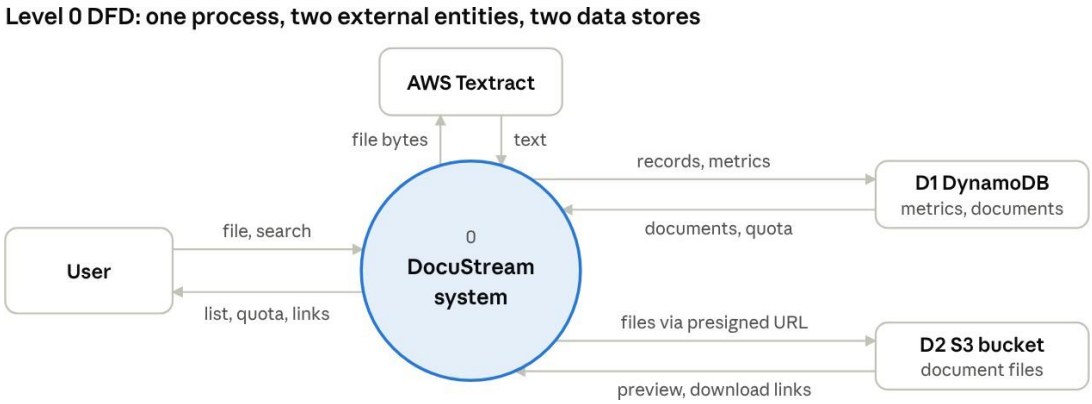
Figure 4.1 – System Architecture

4.2 DATA FLOW DIAGRAM

Level 0 DFD

The user provides a file, a user identity and search text to the DocuStream system, and receives the document list, storage quota, preview and download links, and the extracted OCR text and metadata. The system reads and writes the DynamoDB table (user metrics and document records) and the S3 bucket (document files), and sends file bytes to AWS Textract and receives recognised text in return.

The external entities are the **User** and **AWS Textract**. The data stores are **D1 – DynamoDB table** `DocuStreamData` and **D2 – S3 document bucket**.



level 0 data flow diagram

Figure 4.2 – Data Flow Diagram

4.3 USE CASE DIAGRAM

Actors User and Developer

Use Cases

- Select Demo User Profile
- Upload Document
- View Storage Quota
- View Document List
- Search and Filter Documents
- Preview Document
- Download Document
- View OCR Text and Metadata
- Delete Document
- Run Backend Tests
- Deploy Backend (AWS)
- Deploy Frontend (Vercel)

The user uses the web dashboard to upload, find, inspect and delete documents, while the developer uses the automation scripts to test and deploy the platform.

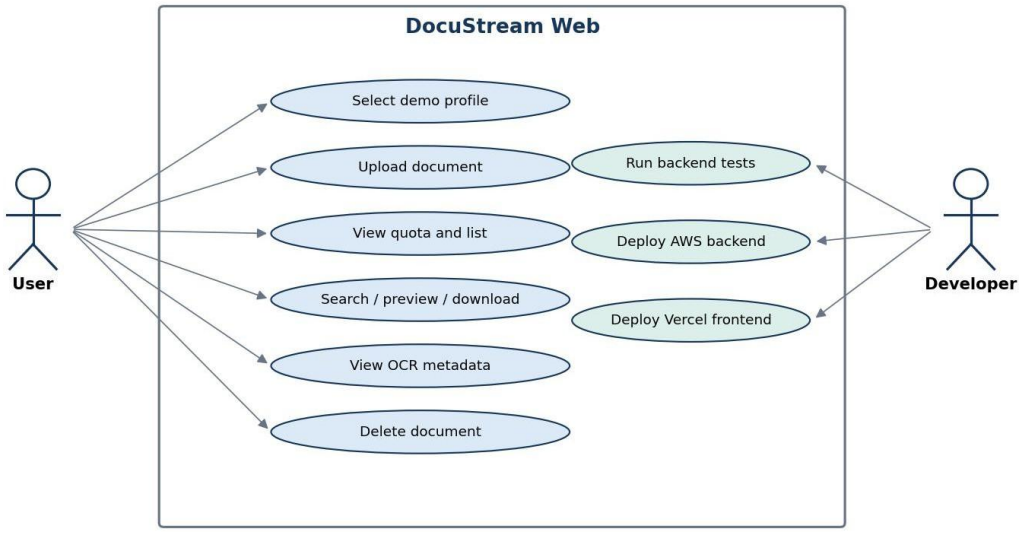


Figure 4.3 – Use Case Diagram

4.4 COMPONENT DIAGRAM

The major components of DocuStream Web are:

- Next.js / React User Interface (header, storage meter, dropzone, search bar, document grid, modal)
- API Client Layer (`frontend/lib/api.ts`) with dynamic base URL
- Local mock API routes (`frontend/app/api/`)
- Amazon API Gateway (`DocuStreamApi`)
- Four Lambda functions: PresignedUrlFunction, DocumentProcessorFunction, DocumentListFunction, DocumentDeleteFunction
- Shared utility module (`src/utils.py`)
- Amazon S3 bucket (`DocuStreamBucket`)
- AWS Textract and the local fallback extractor
- Amazon DynamoDB table (`DocuStreamTable`)
- AWS SAM template (`template.yaml`) that defines all of the above

The frontend communicates with the backend using REST over HTTPS, and with S3 using a direct HTTPS PUT to a presigned URL.

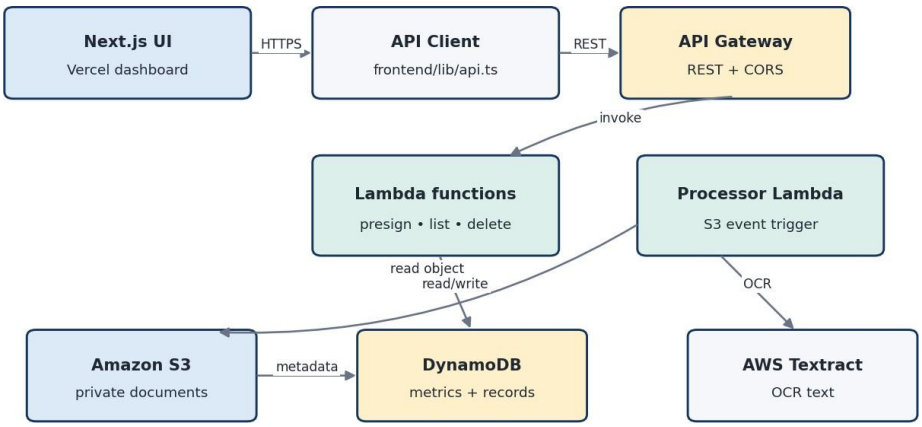


Figure 4.4 – Component Diagram

4.5 SEQUENCE DIAGRAM

The basic sequence from upload to stored result is:

1. The user drops a file on the dashboard; the browser checks its type and size.
2. The browser sends `GET /api/upload-url` with the file name, size, content type and `X-User-Id` header.
3. API Gateway invokes PresignedUrlFunction.
4. The function checks the 20 MB limit, reads the user's `METRICS#USAGE` item, and checks the 100 MB quota.
5. The function generates a document ID, builds the S3 key, writes a `PENDING_UPLOAD` record and returns a 15-minute presigned PUT URL.
6. The browser uploads the file directly to S3 with `PUT` and shows progress.
7. S3 raises an `ObjectCreated` event and invokes DocumentProcessorFunction.
8. The function reads the object, calls Textract `detect_document_text` (or the fallback), and extracts emails, phones and keywords.
9. The function marks the record `PROCESSED` and atomically increments `total_storage_bytes` and `document_count`.
10. The browser polls `GET /api/documents` every 2 seconds; DocumentListFunction returns the records, quota and presigned preview and download URLs.
11. On delete, the browser sends `DELETE /api/documents/{docId}`; DocumentDeleteFunction removes the object and record and reduces the quota.

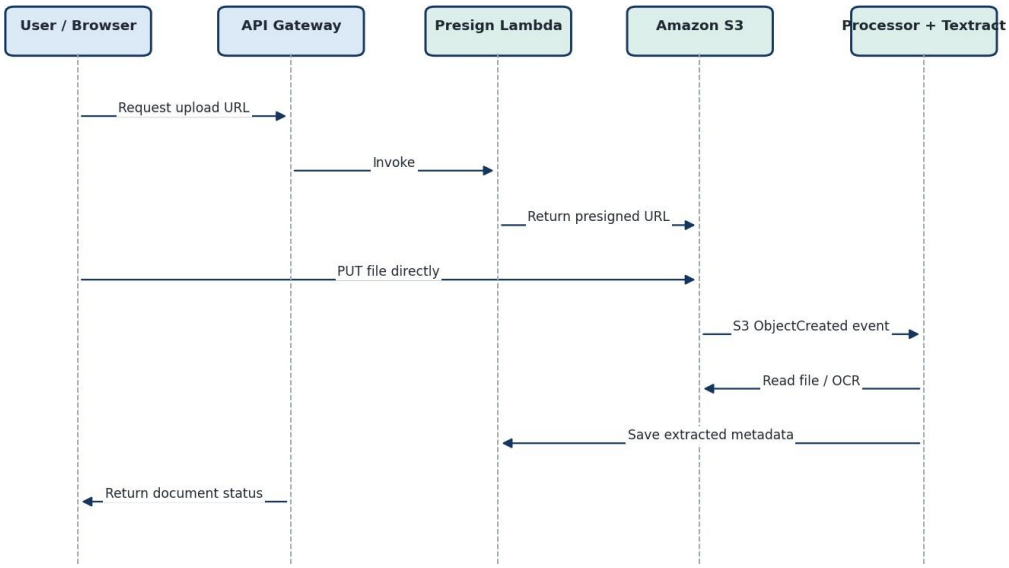


Figure 4.5 – Sequence Diagram

4.6 ACTIVITY DIAGRAM

The activity flow covers file selection, validation, URL issuance, upload, OCR, metadata extraction, storage and display. There are three decision points: whether the file is of an allowed type and size, whether the user has enough quota left, and whether Textract is available (otherwise the local fallback extractor is used).

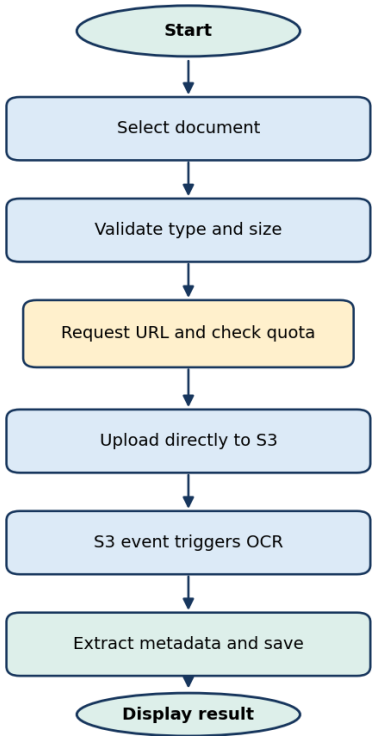


Figure 4.6 – Activity Diagram

4.7 DATA MODEL (CLASS / ENTITY DIAGRAM)

DocuStream uses a single DynamoDB table, `DocuStreamData`, with partition key `partition_key` and sort key `sort_key`. All items for a user share the partition key `USER#<user_id>`. Two entities are stored:

- **UserMetrics** – sort key `METRICS#USAGE`; attributes `user_id`, `total_storage_bytes`, `max_allowed_bytes`, `document_count`, `updated_at`.
- **Document** – sort key `DOC#<doc_id>`; attributes `doc_id`, `user_id`, `file_name`, `original_file_name`, `s3_key`, `s3_bucket`, `file_size_bytes`, `content_type`, `upload_timestamp`, `processed_timestamp`, `processing_status`, `block_count`, `extracted_text`, `parsed_metadata` (emails, phones, keywords).

One user has one UserMetrics item and zero or more Document items.

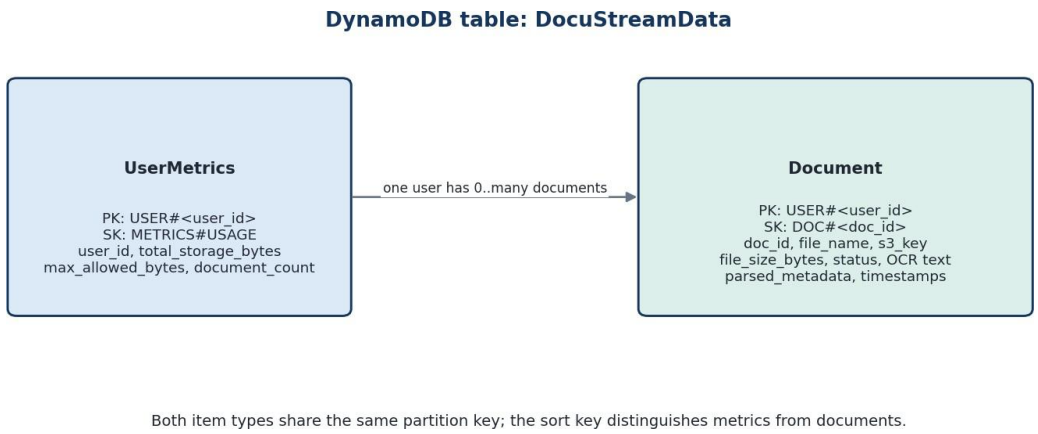


Figure 4.7 – Data Model (Entity Diagra

CHAPTER 5: MODULE DESCRIPTION

DocuStream Web is divided into the following modules. Each module has a clearly defined responsibility, and together they provide the complete document lifecycle.

Table 5.1 – Major Project Modules

Module	File / Location	Responsibility
Presigned URL	src/get_presigned_url.py	Validates file size and quota, creates pending record, issues S3 PUT URL
Document Processing	src/process_document.py	Runs OCR, extracts entities, updates record, increments quota
Document Listing	src/list_documents.py	Queries user partition, creates preview and download URLs, returns quota
Document Deletion	src/delete_document.py	Deletes object and record, reclaims quota
Shared Utilities	src/utils.py	Regular expressions, stop words, JSON encoder, configuration
Infrastructure	template.yaml	SAM definition of S3, DynamoDB, API Gateway, Lambda, IAM
Web Interface	frontend/app frontend/components	Dashboard and state controller
Mock API	frontend/app/api/	Local offline simulation of the backend
Tests	tests/test_backend.py	Eleven pytest/moto unit tests
Deployment	deploy_backend.sh , deploy_frontend.sh	Automated AWS and Vercel deployment

5.1 PRESIGNED URL MODULE

The presigned URL module is the entry point for every upload. It reads `fileName`, `fileSize` and `contentType` from the query string and the user ID from the `X-User-Id` header. It first rejects files larger than 20 MB (`MAX_FILE_BYTES`). It then reads the user's `METRICS#USAGE` item and rejects the request with HTTP 400 if `current storage + file size` would exceed the quota (100 MB by default). The error response contains the figures needed to explain the problem to the user.

If both checks pass, the module generates a UUID v4 document ID, builds the key `uploads/<user_id>/<doc_id>/<file_name>`, writes a `PENDING_UPLOAD` record and returns a presigned PUT URL valid for 900 seconds along with the document metadata. The application server never sees the file bytes.

5.2 DOCUMENT PROCESSING MODULE

The processing module is invoked by the S3 `ObjectCreated` event, not by API Gateway. It parses the object key to recover `user_id`, `doc_id` and `file_name`, reads the object bytes from S3, and calls Textract `detect_document_text`. The `LINE` blocks are joined with newlines to form the extracted text. If Textract raises a client error, such as a missing subscription in an account, the module calls `fallback_extract_text` to read printable text from the byte stream instead of failing.

The text is passed to three extractors: emails (regular expression), phone numbers (regular expression) and keywords (tokenise, lower-case, strip punctuation, remove stop words, rank by frequency, keep the top 15).

The module then updates the document record to `PROCESSED` with the text, metadata and timestamp, and atomically adds the file size and one document to the user's `METRICS#USAGE` item using an `update_item` expression with `if_not_exists`.

5.3 DOCUMENT LISTING MODULE

The listing module queries all items whose partition key is `USER#<user_id>`, separates the metrics item from the document items, and builds the response. For each document it generates two presigned GET URLs with a one-hour expiry. The preview URL sets `ResponseContentDisposition` to `inline` so that the browser renders the file in a tab. The download URL sets it to `attachment` so that the browser saves the file. The response also contains the user's quota metrics, which drive the storage meter.

5.4 DOCUMENT DELETION MODULE

The deletion module first reads the document item to obtain its `s3_key` and `file_size_bytes`. If the item does not exist it returns a not-found response. Otherwise it deletes the S3 object, deletes the DynamoDB record, and runs an `update_item` that subtracts the file size from `total_storage_bytes` and reduces `document_count` by one, so that the user's quota is restored immediately.

5.5 INFRASTRUCTURE MODULE

The `template.yaml` SAM file declares all resources: the S3 bucket with encryption, public access block, CORS and event notification; the DynamoDB table with on-demand billing and point-in-time recovery; the regional REST API; the four Lambda functions with their memory, timeout and IAM policies; and parameters for environment, maximum storage and maximum file size. The template also resolves the circular dependency between the bucket, its notification and the processor function's permissions (Section 6.1).

5.6 WEB USER INTERFACE MODULE

The frontend is a Next.js 14 App Router application. `app/page.tsx` is the central state controller and the following components render the interface:

- **Header** – branding, connection status (API Gateway or Local Edge Mode), demo user switcher and public-demo banner.
- **StorageMeter** – usage bar that turns from green (under 70%) to amber (70–90%) to red (over 90%) with a warning banner.
- **UploadDropzone** – drag-and-drop, type and size validation, and stages `requesting_url`, `uploading_s3`, `processing_textract` and `completed`.
- **SearchFilterBar** – client-side search across file name, OCR text, emails, phones and keywords, with status filter.
- **DocumentList and DocumentCard** – grid with loading skeletons, empty state, and preview, download and delete actions.
- **DocumentModal** – raw OCR text with copy button and chips for emails, phones and keyword

CHAPTER 6: IMPLEMENTATION

6.1 INFRASTRUCTURE AS CODE

The whole backend is declared in `template.yaml` using AWS SAM, an extension of CloudFormation. Three parameters make the stack configurable: `Environment` (dev, staging or prod; default prod), `MaxStorageBytes` (default 104857600, i.e. 100 MB) and `MaxFileBytes` (default 20971520, i.e. 20 MB). A `Globals` section sets the Python 3.11 runtime, x86_64 architecture, a default 30-second timeout and 256 MB memory, and passes the table name, bucket name, limits and environment to every function as environment variables.

Resolving the circular dependency. An S3 bucket that notifies a Lambda function creates a cycle if the function's role also refers to the bucket: the bucket needs the function, the function needs its role, and the role needs the bucket. DocuStream breaks the cycle with a deterministic name and string interpolation:

1. The bucket name is fixed as `docustream-web-storage-${AWS::AccountId}-${AWS::Region}`.
2. The IAM policies and the Lambda permission refer to the bucket by this string (`!Sub`) instead of `!GetAtt DocuStreamBucket.Arn`.
3. The bucket declares `DependsOn: DocumentProcessorPermission`, so that the permission exists before S3 attaches the notification.

Table 6.2 – Lambda Function Configuration

Function	Handler	Memory	Timeout	Trigger	IAM policies
PresignedUrlFunction	src/get_presigned_url.handler	256 MB	30 s	API GET /api/upload-url	DynamoDB CRUD, S3 CRUD
DocumentProcessorFunction	src/process_document.handler	512 MB	90 s	S3 ObjectCreated:* on uploads/	DynamoDB CRUD, S3 read, textract:DetectDocumentText
DocumentListFunction	src/list_documents.handler	256 MB	30 s	API GET /api/documents	DynamoDB read, S3 read
DocumentDeleteFunction	src/delete_document.handler	256 MB	30 s	API DELETE /api/documents/{docId}	DynamoDB CRUD, S3 CRUD

The S3 bucket uses AES256 server-side encryption, blocks all four categories of public access, and allows CORS from `http://localhost:3000` and `https://*.vercel.app` for `GET`, `PUT`, `POST` and `HEAD`. The DynamoDB table uses `PAY_PER_REQUEST` billing, server-side encryption and point-in-time recovery.

6.2 BACKEND IMPLEMENTATION

The backend consists of four handler files and one shared module, all in Python 3.11 with boto3.

src/utils.py holds precompiled regular expressions, the stop-word set, helpers to build API responses, and a `DecimalEncoder` that converts DynamoDB `Decimal` values to JSON numbers. The entity patterns are:

```
EMAIL_REGEX = re.compile(r"[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+",
re.IGNORECASE)
PHONE_REGEX = re.compile(r"(?:\+?\d{1,3}[-.\s]?)?\(?\d{2,4}\)?[-.\s]?\d{3,4}[-
.\s]?\d{3,4}")
```

src/get_presigned_url.py applies the two limits before issuing a URL:

```
if                    file_size                    >                    MAX_FILE_BYTES:
    return build_response(400, {"error": "File size exceeds maximum allowed file limit"})

if          (current_storage          +          file_size)          >          max_quota:
    return build_response(400, {"error": "User storage quota exceeded"})
```

src/process_document.py performs OCR with a fallback and then updates the quota atomically:

```
try:
    resp    =    textract_client.detect_document_text(Document={"Bytes":     file_bytes})
    lines = [b["Text"] for b in resp.get("Blocks", []) if b.get("BlockType") == "LINE"]
    extracted_text                            =                            "\n".join(lines)
except                                                                        ClientError:
    extracted_text                  =                  fallback_extract_text(file_bytes)

table.update_item(
    Key={"partition_key":          user_pk,          "sort_key":          "METRICS#USAGE"},
    UpdateExpression="SET    total_storage_bytes    =    if_not_exists(total_storage_bytes,
:zero)                           +                          :size,                          "
                     "document_count  =  if_not_exists(document_count,  :zero)  +  :one",
    ...
)
```

src/list_documents.py creates the two presigned GET URLs, differing only in `ResponseContentDisposition` (`inline` or `attachment`) and both valid for 3600 seconds.

src/delete_document.py deletes the S3 object and the DynamoDB record and then subtracts `file_size_bytes` from the user's metrics using an atomic `update_item`.

6.3 FRONTEND IMPLEMENTATION

The frontend uses Next.js 14 with the App Router, TypeScript and Tailwind CSS, deployed on Vercel. Its structure is:

- `app/layout.tsx` – root HTML shell, fonts and metadata
- `app/page.tsx` – central reactive state controller (documents, quota, polling, modal)
- `app/api/` – local mock route handlers
- `components/` – Header, StorageMeter, UploadDropzone, SearchFilterBar, DocumentCard, DocumentModal, DocumentList
- `lib/api.ts` – Axios API client
- `lib/types.ts` – TypeScript interfaces and status types

The API client resolves its base URL from `NEXT_PUBLIC_API_GATEWAY_URL`. If the variable is empty, calls go to relative paths and are answered by the local mock routes, which return a mock upload URL, simulate the upload, return sample documents with OCR text and simulate deletion. This allows the interface to be developed and demonstrated without any AWS resources.

```
export             function             getApiBaseUrl():             string             {
  const            envUrl            =            process.env.NEXT_PUBLIC_API_GATEWAY_URL;
  if  (envUrl  &&  envUrl.trim().length  >  0)  return  envUrl.trim().replace(/\/+$/,  "");
  return      "";      //      relative      route:      local      Next.js      mock      API
}
```

During upload, the dropzone validates the file type and size, requests the presigned URL, sends the file to S3 with Axios and tracks progress through `onUploadProgress`. After the upload, the page polls the list endpoint every 2000 ms until the document status becomes `PROCESSED`.

Table 6.1 – REST API Endpoints

Method	Endpoint	Purpose
GET	/api/upload-url?fileName=&fileSize=&contentType=	Validate limits, create pending record, return presigned PUT URL
GET	/api/documents?userId=	List documents with quota, preview and download URLs
DELETE	/api/documents/{docId}?userId=	Delete document and reclaim quota
OPTIONS	/api/{proxy+}	CORS pre-flight

All requests carry the `X-User-Id` header.

6.4 DATA MODEL AND ACCESS PATTERNS

Because a user's metrics item and all document items share one partition key, one `query` call returns the entire state of a user. The access patterns are summarised below.

Table 6.3 – DynamoDB Access Patterns

Business need	Key condition	DynamoDB operation
Validate user quota	partition_key = USER#id and sort_key = METRICS#USAGE	get_item
List all documents of a user	partition_key = USER#id	query
Inspect one document	partition_key = USER#id and sort_key = DOC#doc_id	get_item
Increase usage after processing	metrics key	update_item with if_not_exists(...) + :size
Delete a document and reclaim quota	document key and metrics key	delete_item then update_item

6.5 QUOTA MANAGEMENT

Quota is enforced at two points. The frontend blocks files above 20 MB before any request is made. The presigned URL function then checks the same limit and the 100 MB per-user quota in the backend, so that the rule cannot be bypassed by calling the API directly. Usage is only added to the user's metrics when processing completes, using an atomic `update_item`, and is subtracted at deletion by a matching atomic update. The storage meter in the interface reads these metrics and changes colour as usage approaches the limit.

6.6 SECURITY IMPLEMENTATION

- **Private storage.** All four S3 public-access-block flags are enabled and no object is public; access is only through time-limited SigV4 presigned URLs (15 minutes for upload, one hour for preview and download).
- **Encryption.** S3 uses AES256 (SSE-S3) and DynamoDB uses server-side encryption.
- **Least privilege.** Each Lambda function has its own SAM policy templates scoped to the project's table and bucket. Only the Textract permission uses a wildcard resource, because that action does not support resource-level scoping.
- **User isolation.** Every key is prefixed with `USER#<user_id>` or `uploads/<user_id>/`, so records and objects are separated per user.
- **Abuse protection.** The 20 MB file limit and 100 MB quota limit the cost and storage a public visitor can consume.
- **Repository hygiene.** No credentials are stored in the source; values are resolved through CloudFormation and environment variables.

- **CORS.** S3 CORS is limited to localhost and Vercel origins; API Gateway returns the CORS headers required by the browser client.

6.7 DEPLOYMENT

Backend (`deploy_backend.sh`) verifies the AWS identity with `aws sts get-caller-identity`, runs the unit tests and aborts if they fail, runs `sam build --parallel`, deploys the `docustream-web-stack` CloudFormation stack, reads the stack outputs (API endpoint, bucket name, table name), and writes the API URL into `frontend/.env.local`.

Frontend (`deploy_frontend.sh`) reads the environment file, verifies the production build with `npm run build`, and runs `npx vercel --prod` while passing `NEXT_PUBLIC_API_GATEWAY_URL` as both a build-time and a runtime variable.

The complete lifecycle of a document is summarised in Figure 6.1.

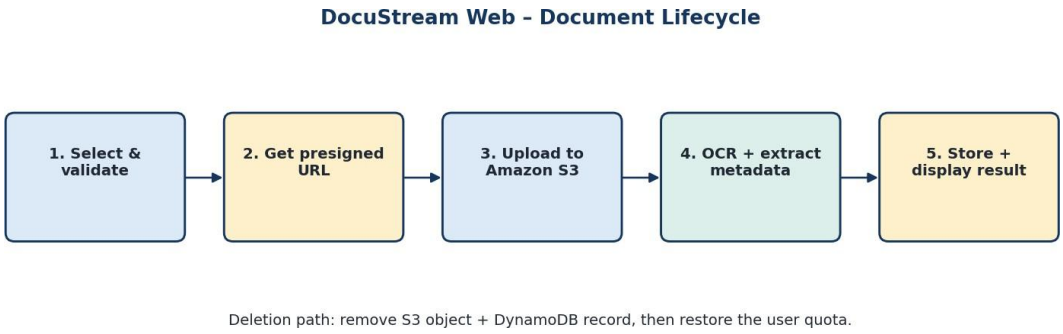


Figure 6.1 – Document Lifecycle

CHAPTER 7: TESTING AND VERIFICATION

Testing verifies that the major components of DocuStream Web work as expected, from upload validation to quota reclamation. Numerical performance figures such as average OCR time, maximum throughput or concurrency are not reported here and should be measured experimentally before being quoted.

7.1 UNIT TESTING

The backend is tested in `tests/test_backend.py` using **pytest** and **moto**, which mock S3, DynamoDB and Textract in memory so that no real AWS resources are used or billed. The suite contains eleven tests, all of which passed in a single run of about 1.1 seconds.

Table 7.1 – Automated Test Cases

Test ID	Test Case	Expected Result	Result
UT01	test_regex_extraction	Complex emails and phone number formats are matched	Pass
UT02	test_keyword_extraction	Stop words and punctuation removed; keywords ranked by frequency	Pass
UT03	test_get_presigned_url_success	Valid request returns a presigned URL and a pending record	Pass
UT04	test_get_presigned_url_file_size_exceeded	File above 20 MB is rejected with HTTP 400	Pass
UT05	test_get_presigned_url_storage_quota_exceeded	Upload that exceeds 100 MB total is rejected with HTTP 400	Pass
UT06	test_process_document_success	Text and metadata extracted; record PROCESSED; quota incremented	Pass
UT07	test_process_document_textract_failure	Textract failure is handled without crashing	Pass
UT08	test_process_document_subscription_required_fallback	Fallback extractor is used when Textract is unavailable	Pass
UT09	test_list_documents	User documents and quota are returned with signed URLs	Pass
UT10	test_delete_document_success	S3 object and record removed; quota restored	Pass
UT11	test_delete_document_not_found	Missing document returns a not-found response	Pass

The deployment script runs this suite before every build and aborts the deployment if any test fails.

7.2 FUNCTIONAL TESTING

The user-facing features are verified through the dashboard against the mock API or the deployed stack. The checklist below lists the functional scenarios; the Result column is to be completed during the final demonstration.

Table 7.2 – Manual Verification Checklist

Test ID	Test Case	Expected Result	Result
FT01	Open dashboard	Header, storage meter, dropzone and document grid are displayed	(record)
FT02	Upload valid PDF	Progress bar completes; document appears as PROCESSED	(record)
FT03	Upload PNG or JPEG	Image is processed and shown on the grid	(record)
FT04	Upload unsupported type	Rejected in the browser with a message	(record)
FT05	Upload file above 20 MB	Rejected before any upload starts	(record)
FT06	Exceed 100 MB total	Backend refuses the URL request; quota warning shown	(record)
FT07	View OCR modal	Extracted text, emails, phones and keywords displayed	(record)
FT08	Search by email or keyword	Only matching documents remain in the grid	(record)
FT09	Preview document	File opens inline in a new browser tab	(record)
FT10	Download document	Browser saves the file	(record)
FT11	Delete document	Card disappears; storage meter decreases	(record)
FT12	Switch demo user	Each user sees only their own documents and quota	(record)

7.3 INTEGRATION TESTING

Integration testing verifies communication between the major components. The following integrations are exercised by the unit tests and by the deployed stack:

- API handler with DynamoDB (quota read, pending record, metrics update)
- Processor function with S3 (object read) and Textract (OCR and fallback)
- Listing function with S3 (presigned URL generation) and DynamoDB (query)
- Deletion function with S3 (object delete) and DynamoDB (record delete, quota update)
- S3 `ObjectCreated` event with the processor Lambda function after deployment
- Next.js frontend with API Gateway through the configured base URL
- Browser direct PUT upload to S3 under the bucket CORS rules

7.4 CLOUD AND SECURITY TESTING

The security checks cover the rules built into the design: oversized files and over-quota uploads are rejected with HTTP 400 (UT04, UT05); the bucket blocks public access so that objects can only be read through presigned URLs; preview and download links expire after one hour and upload links after 15 minutes; each Lambda function holds only the permissions listed in Table 6.2; and user data is separated by the `USER#<user_id>` key prefix.

DocuStream Web is an educational prototype and not a production-grade document platform. Users are chosen from a demo switcher rather than authenticated, so the user separation protects data between demo profiles but not against a user who modifies the request header. Section 8.2 lists this and other limits

CHAPTER 8: CONCLUSION AND FUTURE ENHANCEMENTS

8.1 CONCLUSION

DocuStream Web – A Serverless Document Processing and Metadata Intelligence Platform shows how a complete document intake and OCR pipeline can be built from managed cloud services without running any server. Direct-to-S3 presigned uploads keep file bytes away from the application tier, S3 events trigger processing automatically, Textract performs the OCR, regular expressions and keyword ranking add structured metadata, and DynamoDB stores the results in a single-table design that returns a user's whole state with one query.

The platform adds practical safeguards: a 20 MB file limit, a 100 MB per-user quota with atomic accounting, private encrypted storage, least-privilege IAM and time-limited signed links. The entire stack is defined in one SAM template and deployed with two scripts, and the eleven automated backend tests pass.

The cost and operations profile is the main benefit of the serverless approach, as shown below.

Dimension	Traditional server-based system	DocuStream Web
Idle infrastructure cost	Always-on virtual machines or containers	No charge when there are no requests
Upload bandwidth	Passes through the web server	Direct to S3
Scaling	Auto-scaling groups with warm-up	Automatic Lambda and on-demand DynamoDB scaling
Server maintenance	OS patching and runtime updates	None; managed by AWS

The project provides a clear demonstration of event-driven architecture, object storage, managed AI services, NoSQL data modelling, infrastructure as code, IAM least privilege and automated deployment, and is appropriate for a Cloud Application Development mini project.

8.2 LIMITATIONS

- Users are selected from a demo switcher and identified by the `X-User-Id` header; there is no real authentication, so the system is suitable only as a demonstration.
- Synchronous Textract `detect_document_text` suits images and single-page documents; multi-page PDFs need the asynchronous Textract API.
- The fallback extractor reads printable text from the byte stream and is not a substitute for OCR on scanned images.
- Entity extraction is rule-based: the phone pattern can match other long digit sequences, and keywords are simple frequency counts without semantic understanding.
- Quota is checked when the upload URL is requested but added when processing finishes, so several simultaneous uploads could pass the check before any of them is counted.
- A user who requests an upload URL and never uploads leaves a `PENDING_UPLOAD` record behind.
- The dashboard uses 2-second polling rather than server push, and preview and download links expire after one hour.
- API Gateway returns `Access-Control-Allow-Origin: *`, which is convenient for a public demo but should be restricted in production.
- Costs depend on AWS pricing for Textract, Lambda, S3 and DynamoDB, and are not measured in this report.

8.3 FUTURE ENHANCEMENTS

- Replace demo user selection with Amazon Cognito user pools and JWT authorisation on API Gateway routes.
- Use asynchronous Textract (`start_document_text_detection`) with SQS and SNS to support multi-page and very large PDFs.
- Generate embeddings with Amazon Bedrock and store them in OpenSearch Serverless for semantic search and question answering over documents.
- Add forms and tables extraction with Textract `AnalyzeDocument` for invoices and structured forms.
- Reserve quota when the URL is issued, and add an S3 or DynamoDB lifecycle rule to remove abandoned pending uploads.
- Replace polling with push notifications through WebSockets or server-sent events.
- Restrict API CORS to the production origin and add AWS WAF rate limiting.
- Add CloudWatch dashboards and alarms for errors, duration and Textract usage.
- Add role-based access control, shared workspaces and document retention policies.

CHAPTER 9: REFERENCES

- AWS Serverless Application Model (SAM) Developer Guide. https://docs.aws.amazon.com/serverless-application-model/
- Amazon S3 User Guide – Sharing objects with presigned URLs. https://docs.aws.amazon.com/AmazonS3/latest/userguide/ShareObjectPreSignedURL.html
- Amazon Textract Developer Guide. https://docs.aws.amazon.com/textract/
- AWS Lambda Developer Guide. https://docs.aws.amazon.com/lambda/
- Amazon DynamoDB Developer Guide – Best practices for modelling relational data. https://docs.aws.amazon.com/amazondynamodb/
- Amazon API Gateway Developer Guide. https://docs.aws.amazon.com/apigateway/
- AWS Identity and Access Management User Guide. https://docs.aws.amazon.com/IAM/
- Boto3 Documentation – AWS SDK for Python. https://boto3.amazonaws.com/v1/documentation/api/latest/index.html
- Next.js Documentation – The React framework for the web. https://nextjs.org/docs
- Tailwind CSS Documentation. https://tailwindcss.com/docs
- Vercel Documentation. https://vercel.com/docs
- Moto – Mock AWS Services. https://docs.getmoto.org/
- pytest Documentation. https://docs.pytest.org/
- Project source and README – DocuStream Web.

CHAPTER 10: APPENDIX

A. PROJECT STRUCTURE

```
docustream-web/
├── template.yaml
├── deploy_backend.sh
├── deploy_frontend.sh
├── package.json
├── src/
│   ├── utils.py
│   ├── get_presigned_url.py
│   ├── process_document.py
│   ├── list_documents.py
│   └── delete_document.py
├── tests/
│   └── test_backend.py
└── frontend/
    ├── app/
    │   ├── layout.tsx
    │   ├── page.tsx
    │   ├── globals.css
    │   └── api/ (local mock routes)
    ├── components/
    │   ├── Header.tsx
    │   ├── StorageMeter.tsx
    │   ├── UploadDropzone.tsx
    │   ├── SearchFilterBar.tsx
    │   ├── DocumentList.tsx
    │   ├── DocumentCard.tsx
    │   └── DocumentModal.tsx
    ├── lib/
    │   ├── api.ts
    │   └── types.ts
    └── public/ (static assets)
```

B. IMPORTANT COMMANDS

Install frontend dependencies

```
npm --prefix frontend install
```

Run the web application locally (mock API mode)

```
npm run dev
```

Run the backend tests

```
pytest tests/test_backend.py
```

Configure AWS and deploy the backend

```
aws                                                                    configure
npm run deploy:backend
```

Deploy the frontend to Vercel

```
npm run deploy:frontend
```

C. REPRESENTATIVE DATA RECORDS

User storage metrics item:


```
{
  "partition_key":                                                    "USER#user-demo-1",
  "sort_key":                                                          "METRICS#USAGE",
  "user_id":                                                             "user-demo-1",
  "total_storage_bytes":                                                      6815744,
  "max_allowed_bytes":                                                      104857600,
  "document_count":                                                                 2,
  "updated_at":                                                 "2026-10-14T08:00:03.123456Z"
}
```


Document intelligence item (extracted text shortened):


```
{
  "partition_key":                                                    "USER#user-demo-1",
  "sort_key":                                  "DOC#3f2b4b8a-92f7-4c7a-9a9f-361718318f77",
  "doc_id":                                      "3f2b4b8a-92f7-4c7a-9a9f-361718318f77",
  "file_name":                                      "Senior_Cloud_Architect_Resume.pdf",
  "s3_key":                                 "uploads/user-demo-1/3f2b4b8a-92f7-4c7a-9a9f-
361718318f77/Senior_Cloud_Architect_Resume.pdf",
  "file_size_bytes":                                                          2621440,
  "content_type":                                                      "application/pdf",
  "processing_status":                                                      "PROCESSED",
  "block_count":                                                                 52,
  "parsed_metadata":                                                              {
    "emails":                                                 ["alex.morgan@cloudtech.com"],
    "phones":                              ["+1                              415-555-7890"],
    "keywords":     ["architect",     "serverless",     "solutions",     "experienced"]
  }
}
```

APPENDIX D: SCREENSHOTS

The following screenshots show the DocuStream Web interface and representative document-processing results.

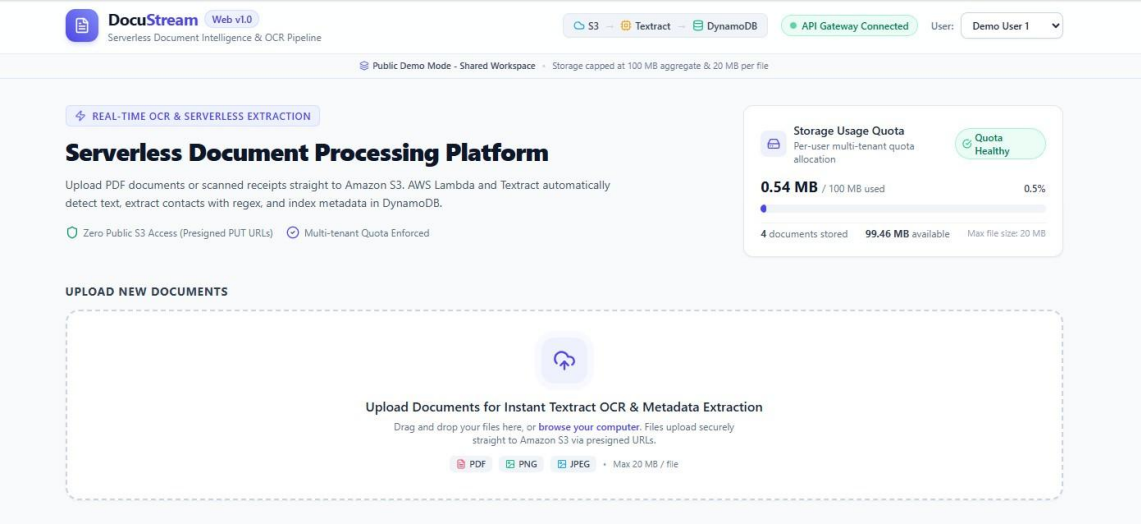


Figure D.1 – DocuStream Web dashboard and upload area

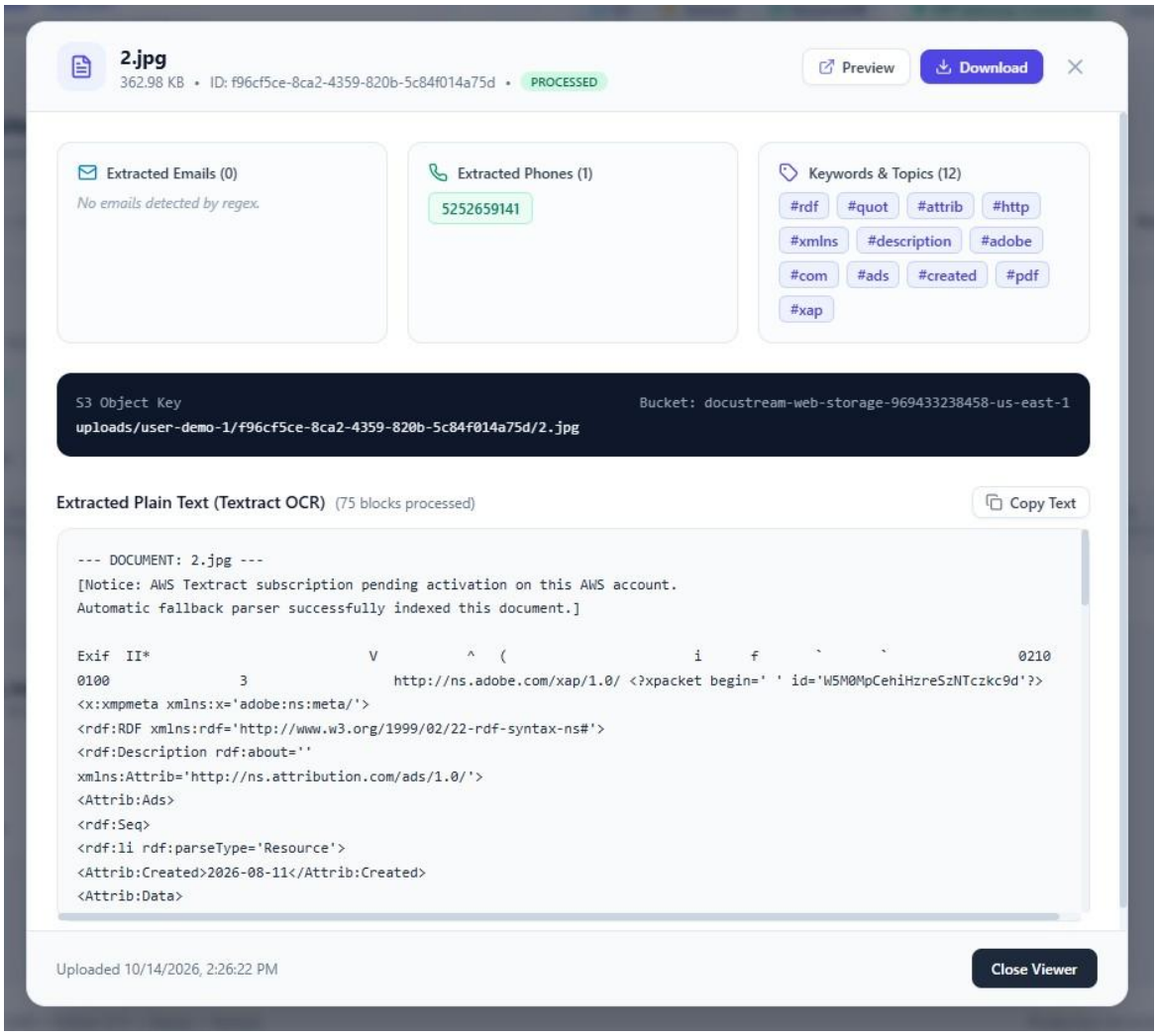


Figure D.2 – Processed image with extracted metadata

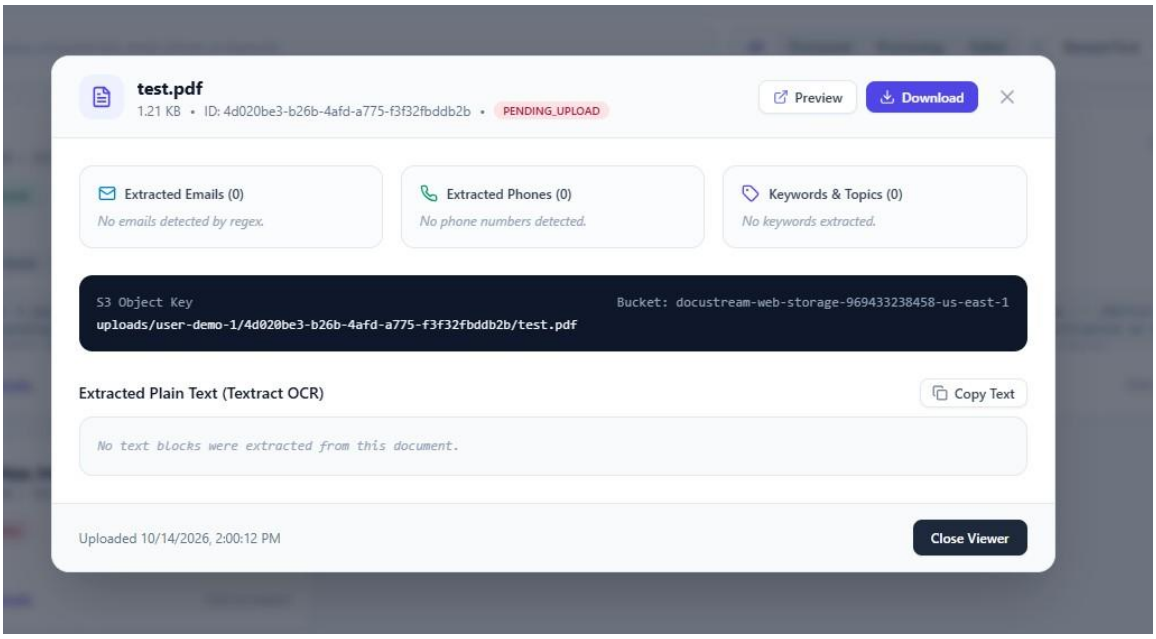


Figure D.3 – Document viewer showing pending upload status

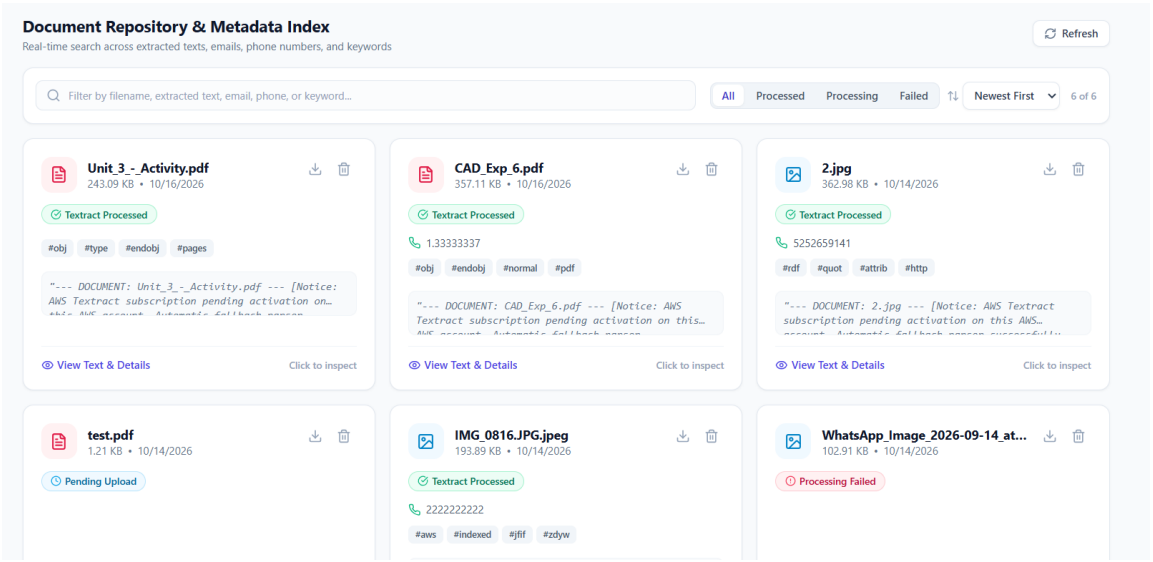


Figure D.4 – OCR text and metadata extraction result